

РАЗРАБОТКА МОДУЛЕЙ

Разработаем 5 модулей в эту структуру:

accounts — авторизация (расширим)

catalog — каталог авто (расширим)

clients — новый модуль клиентов

orders — новый модуль заказов

reports — новый модуль отчётов

```
(.venv) PS C:\Study> python manage.py startapp clients apps/clients  
>> python manage.py startapp orders apps/orders  
>> python manage.py startapp reports apps/reports
```

Обновляем config/settings.py

```
config > settings.py > ...
1  from pathlib import Path
2  import os
3  from dotenv import load_dotenv
4
5  load_dotenv()
6
7  BASE_DIR = Path(__file__).resolve().parent.parent
8
9  SECRET_KEY = os.getenv("SECRET_KEY", "django-insecure-change-me")
10 DEBUG = os.getenv("DEBUG", "True") == "True"
11 ALLOWED_HOSTS = ["*"]
12
13 # ПРИЛОЖЕНИЯ (МОДУЛИ СИСТЕМЫ)
14
15 INSTALLED_APPS = [
16     "django.contrib.admin",
17     "django.contrib.auth",
18     "django.contrib.contenttypes",
19     "django.contrib.sessions",
20     "django.contrib.messages",
21     "django.contrib.staticfiles",
22
23     "apps.accounts", # Модуль 1: Авторизация
24     "apps.catalog", # Модуль 2: Каталог авто
25     "apps.clients", # Модуль 3: Клиенты
26     "apps.orders", # Модуль 4: Заказы
27     "apps.reports", # Модуль 5: Отчёты
28 ]
29
30 MIDDLEWARE = [
31     "django.middleware.security.SecurityMiddleware",
32     "django.contrib.sessions.middleware.SessionMiddleware",
33     "django.middleware.common.CommonMiddleware",
34     "django.middleware.csrf.CsrfViewMiddleware",
35     "django.contrib.auth.middleware.AuthenticationMiddleware",
36     "django.contrib.messages.middleware.MessageMiddleware",
37     "django.middleware.clickjacking.XFrameOptionsMiddleware",
38 ]
39
40 ROOT_URLCONF = "config.urls"
41
42 TEMPLATES = [
43     {
44         "BACKEND": "django.template.backends.django.DjangoTemplates",
45         "DIRS": [BASE_DIR / "templates"],
46         "APP_DIRS": True,
47         "OPTIONS": {
48             "context_processors": [
```

```

46         "APP_DIRS": True,
47         "OPTIONS": {
48             "context_processors": [
49                 "django.template.context_processors.debug",
50                 "django.template.context_processors.request",
51                 "django.contrib.auth.context_processors.auth",
52                 "django.contrib.messages.context_processors.messages",
53             ],
54         },
55     },
56 ]
57
58 WSGI_APPLICATION = "config.wsgi.application"
59
60 # БАЗА ДАННЫХ (PostgreSQL)
61
62 DATABASES = {
63     "default": {
64         "ENGINE": "django.db.backends.postgresql",
65         "NAME": os.getenv("DB_NAME", "autosalon_db"),
66         "USER": os.getenv("DB_USER", "autosalon"),
67         "PASSWORD": os.getenv("DB_PASSWORD", "autosalon_pass"),
68         "HOST": os.getenv("DB_HOST", "localhost"),
69         "PORT": os.getenv("DB_PORT", "5432"),
70     }
71 }
72
73
74 # АУТЕНТИФИКАЦИЯ
75
76 AUTH_USER_MODEL = "accounts.User" # ← наша кастомная модель
77
78 AUTH_PASSWORD_VALIDATORS = [
79     {"NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},
80     {"NAME": "django.contrib.auth.password_validation.MinimumLengthValidator"},
81     {"NAME": "django.contrib.auth.password_validation.CommonPasswordValidator"},
82     {"NAME": "django.contrib.auth.password_validation.NumericPasswordValidator"},
83 ]
84
85 LANGUAGE_CODE = "ru-ru"
86 TIME_ZONE = "Europe/Moscow"
87 USE_I18N = True
88 USE_TZ = True
89
90 STATIC_URL = "static/"
91 STATICFILES_DIRS = [BASE_DIR / "static"]
92 STATIC_ROOT = BASE_DIR / "staticfiles"
93

```

```

90 STATIC_URL = "static/"
91 STATICFILES_DIRS = [BASE_DIR / "static"]
92 STATIC_ROOT = BASE_DIR / "staticfiles"
93
94 MEDIA_URL = "media/"
95 MEDIA_ROOT = BASE_DIR / "media"
96
97 DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
98
99 # Логин/логаут
100 LOGIN_URL = "/accounts/login/"
101 LOGIN_REDIRECT_URL = "/"
102 LOGOUT_REDIRECT_URL = "/accounts/login/"

```

МОДУЛЬ 1: АВТОРИЗАЦИЯ

apps/accounts/models.py

```

apps > accounts > models.py > User > __str__
1  from django.contrib.auth.models import AbstractUser
2  from django.db import models
3
4
5  class User(AbstractUser):
6      """Кастомный пользователь с должностью."""
7
8      class Position(models.TextChoices):
9          ADMIN = "admin", "Администратор"
10         MANAGER = "manager", "Менеджер"
11         DIRECTOR = "director", "Директор"
12
13         phone = models.CharField("Телефон", max_length=20, blank=True)
14         position = models.CharField(
15             "Должность", max_length=20,
16             choices=Position.choices, default=Position.MANAGER,
17         )
18
19         class Meta:
20             verbose_name = "Сотрудник"
21             verbose_name_plural = "Сотрудники"
22
23         def __str__(self):
24             return f"{self.get_full_name() or self.username} ({self.get_position_display()})"

```

apps/accounts/forms.py – создать

```

apps > accounts > forms.py > RegisterForm > save
1  """МОДУЛЬ 1 (АВТОРИЗАЦИЯ): Формы. Формы для входа и регистрации."""
2
3  from django import forms
4  from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
5  from .models import User
6
7
8  class LoginForm(AuthenticationForm):
9      """Форма входа. Стандартная, но с русскими подписями."""
10     username = forms.CharField(
11         label="Email или логин",
12         widget=forms.TextInput(attrs={"class": "form-control", "placeholder": "Введите email"}),
13     )
14     password = forms.CharField(
15         label="Пароль",
16         widget=forms.PasswordInput(attrs={"class": "form-control", "placeholder": "Введите пароль"}),
17     )
18
19
20 class RegisterForm(UserCreationForm):
21     """Форма регистрации нового сотрудника."""
22     full_name = forms.CharField(label="ФИО", max_length=150)
23     email = forms.EmailField(label="Email")
24     phone = forms.CharField(label="Телефон", max_length=20, required=False)
25
26     class Meta:
27         model = User
28         fields = ("username", "full_name", "email", "phone", "position")
29
30     def save(self, commit=True):
31         """Сохраняем full_name в first_name/last_name."""
32         user = super().save(commit=False)
33         parts = self.cleaned_data["full_name"].split(maxsplit=1)
34         user.first_name = parts[0] if parts else ""
35         user.last_name = parts[1] if len(parts) > 1 else ""
36         if commit:
37             user.save()
38         return user

```

apps/accounts/views.py

apps > accounts > views.py > MeView > get

```
1  from rest_framework import status
2  from rest_framework.views import APIView
3  from rest_framework.response import Response
4  from rest_framework.permissions import AllowAny, IsAuthenticated
5  from rest_framework_simplejwt.tokens import RefreshToken
6
7  from .serializers import RegisterSerializer, LoginSerializer, UserSerializer
8
9
10 class RegisterView(APIView):
11     """POST /api/auth/register/ – регистрация."""
12     permission_classes = [AllowAny]
13
14     def post(self, request):
15         serializer = RegisterSerializer(data=request.data)
16         serializer.is_valid(raise_exception=True)
17         user = serializer.save()
18
19         # Сразу выдаём токены
20         refresh = RefreshToken.for_user(user)
21         return Response({
22             "user": UserSerializer(user).data,
23             "access": str(refresh.access_token),
24             "refresh": str(refresh),
25         }, status=status.HTTP_201_CREATED)
26
27
28 class LoginView(APIView):
29     """POST /api/auth/login/ – вход."""
30     permission_classes = [AllowAny]
31
32     def post(self, request):
33         serializer = LoginSerializer(data=request.data)
34         serializer.is_valid(raise_exception=True)
35         user = serializer.validated_data["user"]
36
37         refresh = RefreshToken.for_user(user)
38         return Response({
39             "user": UserSerializer(user).data,
40             "access": str(refresh.access_token),
41             "refresh": str(refresh),
42         })
43
44
45 class MeView(APIView):
46     """GET /api/auth/me/ – текущий пользователь."""
47     permission_classes = [IsAuthenticated]
48
49     def get(self, request):
50         return Response(UserSerializer(request.user).data)
```

apps/accounts/urls.py

```
apps > accounts > urls.py > ...
1  from django.urls import path
2  from rest_framework_simplejwt.views import TokenRefreshView
3  from . import views
4
5  app_name = "accounts"
6
7  urlpatterns = [
8      path("register/", views.RegisterView.as_view(), name="register"),
9      path("login/", views.LoginView.as_view(), name="login"),
10     path("me/", views.MeView.as_view(), name="me"),
11     path("token/refresh/", TokenRefreshView.as_view(), name="token_refresh"),
12 ]
```

apps/accounts/admin.py

```
apps > accounts > admin.py > UserAdmin > get_full_name
1  from django.contrib import admin
2  from django.contrib.auth.admin import UserAdmin as BaseUserAdmin
3  from .models import User
4
5
6  @admin.register(User)
7  class UserAdmin(BaseUserAdmin):
8      list_display = ("username", "email", "get_full_name", "position", "is_active")
9      list_filter = ("position", "is_active", "is_staff")
10     search_fields = ("username", "email", "first_name", "last_name")
11
12     fieldsets = BaseUserAdmin.fieldsets + (
13         ("Дополнительно", {"fields": ("phone", "position")}),
14     )
15
16     @admin.display(description="ФИО")
17     def get_full_name(self, obj):
18         return obj.get_full_name() or "-"
```

МОДУЛЬ 2: КАТАЛОГ АВТО (apps/catalog/)

apps/catalog/models.py

apps > catalog > models.py > ...

```
1 from django.db import models
2
3
4 class Brand(models.Model):
5     """Марка авто."""
6     name = models.CharField("Название", max_length=50, unique=True)
7     country = models.CharField("Страна", max_length=50, blank=True)
8     created_at = models.DateTimeField(auto_now_add=True)
9
10    class Meta:
11        verbose_name = "Марка"
12        verbose_name_plural = "Марки"
13        ordering = ["name"]
14
15    def __str__(self):
16        return self.name
17
18
19 class Car(models.Model):
20     """Автомобиль."""
21
22    class Status(models.TextChoices):
23        AVAILABLE = "available", "В наличии"
24        RESERVED = "reserved", "Забронирован"
25        SOLD = "sold", "Продан"
26
27    class FuelType(models.TextChoices):
28        PETROL = "petrol", "Бензин"
29        DIESEL = "diesel", "Дизель"
30        ELECTRO = "electro", "Электро"
31        HYBRID = "hybrid", "Гибрид"
32
33    brand = models.ForeignKey(
34        Brand, on_delete=models.PROTECT,
35        related_name="cars", verbose_name="Марка",
36    )
37    model = models.CharField("Модель", max_length=100)
38    year = models.PositiveIntegerField("Год")
39    price = models.DecimalField("Цена, ₽", max_digits=12, decimal_places=2)
40    color = models.CharField("Цвет", max_length=30, blank=True)
41    engine_volume = models.DecimalField(
42        "Объём, л", max_digits=3, decimal_places=1, null=True, blank=True,
43    )
44    fuel_type = models.CharField(
45        "Топливо", max_length=20, choices=FuelType.choices, blank=True,
46    )
47    transmission = models.CharField("КПП", max_length=20, blank=True)
48    mileage = models.PositiveIntegerField("Пробег, км", default=0)
49    status = models.CharField(
50        "Статус", max_length=20,
51        choices=Status.choices, default=Status.AVAILABLE,
52    )
53    vin = models.CharField("VIN", max_length=17, unique=True, blank=True, null=True)
54    description = models.TextField("Описание", blank=True)
55    created_at = models.DateTimeField(auto_now_add=True)
56
57    class Meta:
58        verbose_name = "Автомобиль"
59        verbose_name_plural = "Автомобили"
60        ordering = ["-created_at"]
61
62    def __str__(self):
63        return f"{self.brand.name} {self.model} ({self.year})"
```

```

62
63     class Meta:
64         verbose_name = "Автомобиль"
65         verbose_name_plural = "Автомобили"
66         ordering = ["-created_at"]
67
68     def __str__(self):
69         return f"{self.brand.name} {self.model} ({self.year})"
70
71     def get_absolute_url(self):
72         return reverse("catalog:car_detail", args=[self.pk])
73
74     @property
75     def is_available(self):
76         return self.status == self.Status.AVAILABLE

```

apps/catalog/views.py

```

apps > catalog > views.py > CarViewSet
1  """ViewSet автоматически создаёт все CRUD-эндпоинты:
2      GET    /api/cars/          - список
3      POST   /api/cars/          - создать
4      GET    /api/cars/{id}/     - детали
5      PUT    /api/cars/{id}/     - обновить
6      DELETE /api/cars/{id}/     - удалить"""
7
8  from rest_framework import viewsets, filters
9  from django_filters.rest_framework import DjangoFilterBackend
10 from .models import Brand, Car
11 from .serializers import BrandSerializer, CarSerializer
12
13
14 class BrandViewSet(viewsets.ModelViewSet):
15     """CRUD для марок."""
16     queryset = Brand.objects.all()
17     serializer_class = BrandSerializer
18     filter_backends = [filters.SearchFilter]
19     search_fields = ["name", "country"]
20
21
22 class CarViewSet(viewsets.ModelViewSet):
23     """CRUD для авто с фильтрами."""
24     queryset = Car.objects.select_related("brand").all()
25     serializer_class = CarSerializer
26     filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.OrderingFilter]
27     filterset_fields = ["brand", "status", "fuel_type", "year"]
28     search_fields = ["model", "vin", "brand_name"]
29     ordering_fields = ["price", "year", "created_at"]

```

apps/catalog/urls.py

```

apps > catalog > urls.py > ...
1  from django.urls import path, include
2  from rest_framework.routers import DefaultRouter
3  from .views import BrandViewSet, CarViewSet
4
5  router = DefaultRouter()
6  router.register("brands", BrandViewSet, basename="brands")
7  router.register("cars", CarViewSet, basename="cars")
8
9  urlpatterns = [
10 |     path("", include(router.urls)),
11 | ]

```

apps/catalog/admin.py

```

apps > catalog > admin.py > CarAdmin
1  from django.contrib import admin
2  from .models import Brand, Car
3
4
5  @admin.register(Brand)
6  class BrandAdmin(admin.ModelAdmin):
7  |     list_display = ("name", "country", "created_at")
8  |     search_fields = ("name",)
9
10
11 @admin.register(Car)
12 class CarAdmin(admin.ModelAdmin):
13 |     list_display = ("brand", "model", "year", "price", "status", "mileage")
14 |     list_filter = ("brand", "status", "fuel_type", "year")
15 |     search_fields = ("model", "vin")
16 |     list_editable = ("status",)

```

МОДУЛЬ 3: КЛИЕНТЫ (apps/clients/)

apps/clients/models.py

```

apps > clients > models.py > Client > __str__
1  from django.db import models
2  from django.conf import settings
3
4
5  class Client(models.Model):
6      """Клиент автосалона."""
7      full_name = models.CharField("ФИО", max_length=100)
8      phone = models.CharField("Телефон", max_length=20, unique=True)
9      email = models.EmailField("Email", blank=True)
10     passport = models.CharField("Паспорт", max_length=20, blank=True)
11     birth_date = models.DateField("Дата рождения", null=True, blank=True)
12     manager = models.ForeignKey(
13         settings.AUTH_USER_MODEL,
14         on_delete=models.SET_NULL,
15         null=True, blank=True,
16         related_name="clients",
17         verbose_name="Менеджер",
18     )
19     created_at = models.DateTimeField(auto_now_add=True)
20
21     class Meta:
22         verbose_name = "Клиент"
23         verbose_name_plural = "Клиенты"
24         ordering = ["-created_at"]
25
26     def __str__(self):
27         return f"{self.full_name} ({self.phone})"

```

apps/clients/forms.py

```

apps > clients > forms.py > ClientForm > Meta
1  """МОДУЛЬ 3 (КЛИЕНТЫ): Формы."""
2
3  from django import forms
4  from .models import Client
5
6
7  class ClientForm(forms.ModelForm):
8      """Форма создания/редактирования клиента."""
9
10     class Meta:
11         model = Client
12         fields = ["full_name", "phone", "email", "passport", "birth_date", "manager"]
13         widgets = {
14             "full_name": forms.TextInput(attrs={"class": "form-control"}),
15             "phone": forms.TextInput(attrs={"class": "form-control"}),
16             "email": forms.EmailInput(attrs={"class": "form-control"}),
17             "passport": forms.TextInput(attrs={"class": "form-control"}),
18             "birth_date": forms.DateInput(attrs={"class": "form-control", "type": "date"}),
19             "manager": forms.Select(attrs={"class": "form-select"}),
20         }

```

apps/clients/views.py

```
apps > clients > views.py > ClientViewSet
1  from rest_framework import viewsets, filters
2  from .models import Client
3  from .serializers import ClientSerializer
4
5
6  class ClientViewSet(viewsets.ModelViewSet):
7      """CRUD для клиентов."""
8      queryset = Client.objects.select_related("manager").all()
9      serializer_class = ClientSerializer
10     filter_backends = [filters.SearchFilter, filters.OrderingFilter]
11     search_fields = ["full_name", "phone", "email"]
12     ordering_fields = ["created_at", "full_name"]
```

apps/clients/urls.py

```
apps > clients > urls.py > ...
1  from django.urls import path, include
2  from rest_framework.routers import DefaultRouter
3  from .views import ClientViewSet
4
5  router = DefaultRouter()
6  router.register("", ClientViewSet, basename="clients")
7
8  urlpatterns = [
9      path("", include(router.urls)),
10 ]
```

apps/clients/admin.py

```
apps > clients > admin.py > ClientAdmin
1  from django.contrib import admin
2  from .models import Client
3
4
5  @admin.register(Client)
6  class ClientAdmin(admin.ModelAdmin):
7      list_display = ("full_name", "phone", "email", "manager", "created_at")
8      search_fields = ("full_name", "phone")
9      list_filter = ("manager", "created_at")
```

МОДУЛЬ 4: ЗАКАЗЫ

apps/orders/models.py

apps > orders > models.py > ...

```
1 from django.db import models
2 from django.conf import settings
3 from django.utils import timezone
4 from apps.catalog.models import Car
5 from apps.clients.models import Client
6
7
8 class Order(models.Model):
9     """Заказ: покупка, тест-драйв, сервис."""
10
11     class OrderType(models.TextChoices):
12         PURCHASE = "purchase", "Покупка"
13         TEST_DRIVE = "test_drive", "Тест-драйв"
14         SERVICE = "service", "Сервис"
15
16     class Status(models.TextChoices):
17         NEW = "new", "Новый"
18         CONFIRMED = "confirmed", "Подтверждён"
19         COMPLETED = "completed", "Завершён"
20         CANCELLED = "cancelled", "Отменён"
21
22     client = models.ForeignKey(
23         Client, on_delete=models.CASCADE,
24         related_name="orders", verbose_name="Клиент",
25     )
26     car = models.ForeignKey(
27         Car, on_delete=models.CASCADE, (function) verbose_name: Any
28         related_name="orders", verbose_name="Автомобиль",
29     )
30     manager = models.ForeignKey(
31         settings.AUTH_USER_MODEL, on_delete=models.PROTECT,
32         related_name="orders", verbose_name="Менеджер",
33     )
34     order_type = models.CharField("Тип", max_length=20, choices=OrderType.choices)
35     total_amount = models.DecimalField("Сумма, ₽", max_digits=12, decimal_places=2, default=0)
36     status = models.CharField("Статус", max_length=20, choices=Status.choices, default=Status.NEW)
37     comment = models.TextField("Комментарий", blank=True)
38     created_at = models.DateTimeField("Создан", auto_now_add=True)
39     completed_at = models.DateTimeField("Завершён", null=True, blank=True)
40
41     class Meta:
42         verbose_name = "Заказ"
43         verbose_name_plural = "Заказы"
44         ordering = ["-created_at"]
45
46     def __str__(self):
47         return f"Заказ №{self.pk} — {self.client.full_name}"
48
49     def save(self, *args, **kwargs):
50         """
51         Каскад: при создании purchase — авто резервируется.
52         """
53         if not self.pk and self.order_type == self.OrderType.PURCHASE:
54             if self.car.status == Car.Status.AVAILABLE:
55                 self.car.status = Car.Status.RESERVED
56                 self.car.save()
57         super().save(*args, **kwargs)
58
59     def complete(self):
60         """Завершить заказ каскадом."""
61         self.status = self.Status.COMPLETED
62         self.completed_at = timezone.now()
63         if self.order_type == self.OrderType.PURCHASE:
64             self.car.status = Car.Status.SOLD
65             self.car.save()
66         self.save()
```

```

60         """Завершить заказ каскадом."""
61         self.status = self.Status.COMPLETED
62         self.completed_at = timezone.now()
63         if self.order_type == self.OrderType.PURCHASE:
64             self.car.status = Car.Status.SOLD
65             self.car.save()
66         self.save()
67
68     def cancel(self):
69         """Отменить заказ, вернуть авто в available."""
70         self.status = self.Status.CANCELLED
71         if self.car.status == Car.Status.RESERVED:
72             self.car.status = Car.Status.AVAILABLE
73             self.car.save()
74         self.save()

```

apps/orders/forms.py

```

apps > orders > forms.py > OrderForm > __init__
1     """МОДУЛЬ 4 (ЗАКАЗЫ): Формы."""
2
3     from django import forms
4     from .models import Order
5
6
7     class OrderForm(forms.ModelForm):
8         """Форма создания заказа."""
9
10        class Meta:
11            model = Order
12            fields = ["client", "car", "order_type", "total_amount", "comment"]
13            widgets = {
14                "client": forms.Select(attrs={"class": "form-select"}),
15                "car": forms.Select(attrs={"class": "form-select"}),
16                "order_type": forms.Select(attrs={"class": "form-select"}),
17                "total_amount": forms.NumberInput(attrs={"class": "form-control"}),
18                "comment": forms.Textarea(attrs={"class": "form-control", "rows": 3}),
19            }
20
21        def __init__(self, *args, **kwargs):
22            super().__init__(*args, **kwargs)
23            # Показываем только доступные авто
24            from apps.catalog.models import Car
25            self.fields["car"].queryset = Car.objects.filter(status=Car.Status.AVAILABLE)

```

apps/orders/views.py

```
apps > orders > urls.py > ...
1  from django.urls import path, include
2  from rest_framework.routers import DefaultRouter
3  from .views import OrderViewSet
4
5  router = DefaultRouter()
6  router.register("", OrderViewSet, basename="orders")
7
8  urlpatterns = [
9      path("", include(router.urls)),
10 ]
```

apps/orders/urls.py

```
apps > orders > urls.py > ...
1  """МОДУЛЬ 4 (ЗАКАЗЫ): URL-маршруты."""
2
3  from django.urls import path
4  from . import views
5
6  app_name = "orders"
7
8  urlpatterns = [
9      path("", views.order_list, name="order_list"),
10     path("create/", views.order_create, name="order_create"),
11     path("<int:pk>/", views.order_detail, name="order_detail"),
12     path("<int:pk>/complete/", views.order_complete, name="order_complete"),
13     path("<int:pk>/cancel/", views.order_cancel, name="order_cancel"),
14 ]
```

apps/orders/admin.py

```
apps > orders > admin.py > OrderAdmin
1  """МОДУЛЬ 4 (ЗАКАЗЫ): Админка."""
2
3  from django.contrib import admin
4  from .models import Order
5
6
7  @admin.register(Order)
8  class OrderAdmin(admin.ModelAdmin):
9      list_display = ("id", "client", "car", "order_type", "total_amount", "status", "created_at")
10     list_filter = ("status", "order_type", "created_at")
11     search_fields = ("client__full_name", "car__model")
12     readonly_fields = ("created_at", "completed_at")
```

МОДУЛЬ 5: ОТЧЁТЫ (apps/reports/)

apps/reports/services.py

apps > reports > services.py > ...

```
1 from django.db.models import Sum, Count
2 from django.utils import timezone
3 from datetime import timedelta
4 from apps.orders.models import Order
5 from apps.catalog.models import Car
6
7
8 class ReportService:
9     """Сервис отчётов."""
10
11     @staticmethod
12     def kpi(days: int = 30) -> dict:
13         """Главные KPI."""
14         since = timezone.now() - timedelta(days=days)
15         completed = Order.objects.filter(
16             order_type=Order.OrderType.PURCHASE,
17             status=Order.Status.COMPLETED,
18             created_at__gte=since,
19         )
20         revenue = completed.aggregate(total=Sum("total_amount"))["total"] or 0
21         count = completed.count()
22         return {
23             "revenue": float(revenue),
24             "sales_count": count,
25             "avg_check": float(revenue) / count if count else 0,
26             "in_stock": Car.objects.filter(status=Car.Status.AVAILABLE).count(),
27             "period_days": days,
28         }
29
30     @staticmethod
31     def revenue_by_day(days: int = 30) -> list:
32         """Выручка по дням."""
33         since = timezone.now() - timedelta(days=days)
34         rows = (
35             Order.objects.filter(
36                 order_type=Order.OrderType.PURCHASE,
37                 status=Order.Status.COMPLETED,
38                 created_at__gte=since,
39             )
40             .extra(select={"day": "DATE(created_at)"})
41             .values("day")
42             .annotate(total=Sum("total_amount"))
43             .order_by("day")
44         )
45         return [{"date": str(r["day"]), "total": float(r["total"])} for r in rows]
46
47     @staticmethod
48     def top_models(limit: int = 5) -> list:
49         """Топ моделей."""
50         rows = (
51             Order.objects.filter(
52                 order_type=Order.OrderType.PURCHASE,
53                 status=Order.Status.COMPLETED,
54             )
55             .values("car__brand__name", "car__model")
56             .annotate(count=Count("id"))
57             .order_by("-count")[:limit]
58         )
59         return [
60             {"brand": r["car__brand__name"], "model": r["car__model"], "count": r["count"]}
61             for r in rows
62         ]
63
64     @staticmethod
65     def manager_load(days: int = 30) -> list:
66         """Загрузка менеджеров."""
67         since = timezone.now() - timedelta(days=days)
```

```

67         since = timezone.now() - timedelta(days=days)
68         rows = (
69             Order.objects.filter(
70                 order_type=Order.OrderType.PURCHASE,
71                 created_at__gte=since,
72             )
73             .values("manager__username")
74             .annotate(count=Count("id"), total=Sum("total_amount"))
75         )
76         return [
77             {"manager": r["manager__username"], "orders": r["count"], "total": float(r["total"] or 0)}
78             for r in rows
79         ]
80
81     @staticmethod
82     def stock_status() -> list:
83         """Состояние склада."""
84         return [
85             {"status": r["status"], "count": r["count"]}
86             for r in Car.objects.values("status").annotate(count=Count("id"))
87         ]

```

apps/reports/views.py

```

apps > reports > views.py > StockAPIView > get
1  from rest_framework.views import APIView
2  from rest_framework.response import Response
3  from .services import ReportService
4
5
6  class KPIAPIView(APIView):
7      """GET /api/reports/kpi/?days=30"""
8      def get(self, request):
9          days = int(request.GET.get("days", 30))
10         return Response(ReportService.kpi(days))
11
12
13  class RevenueByDayAPIView(APIView):
14      """GET /api/reports/revenue/"""
15      def get(self, request):
16          days = int(request.GET.get("days", 30))
17          return Response(ReportService.revenue_by_day(days))
18
19
20  class TopModelsAPIView(APIView):
21      """GET /api/reports/top-models/"""
22      def get(self, request):
23          limit = int(request.GET.get("limit", 5))
24          return Response(ReportService.top_models(limit))
25
26
27  class ManagerLoadAPIView(APIView):
28      """GET /api/reports/manager-load/"""
29      def get(self, request):
30          days = int(request.GET.get("days", 30))
31          return Response(ReportService.manager_load(days))
32
33
34  class StockAPIView(APIView):
35      """GET /api/reports/stock/"""
36      def get(self, request):
37          return Response(ReportService.stock_status())

```

apps/reports/urls.py

```

apps > reports > urls.py > ...
1  from django.urls import path
2  from . import views
3
4  app_name = "reports"
5
6  urlpatterns = [
7      path("kpi/", views.KPIAPIView.as_view(), name="kpi"),
8      path("revenue/", views.RevenueByDayAPIView.as_view(), name="revenue"),
9      path("top-models/", views.TopModelsAPIView.as_view(), name="top_models"),
10     path("manager-load/", views.ManagerLoadAPIView.as_view(), name="manager_load"),
11     path("stock/", views.StockAPIView.as_view(), name="stock"),
12 ]

```

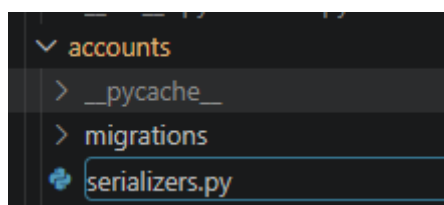
ИНТЕГРАЦИЯ: config/urls.py

```

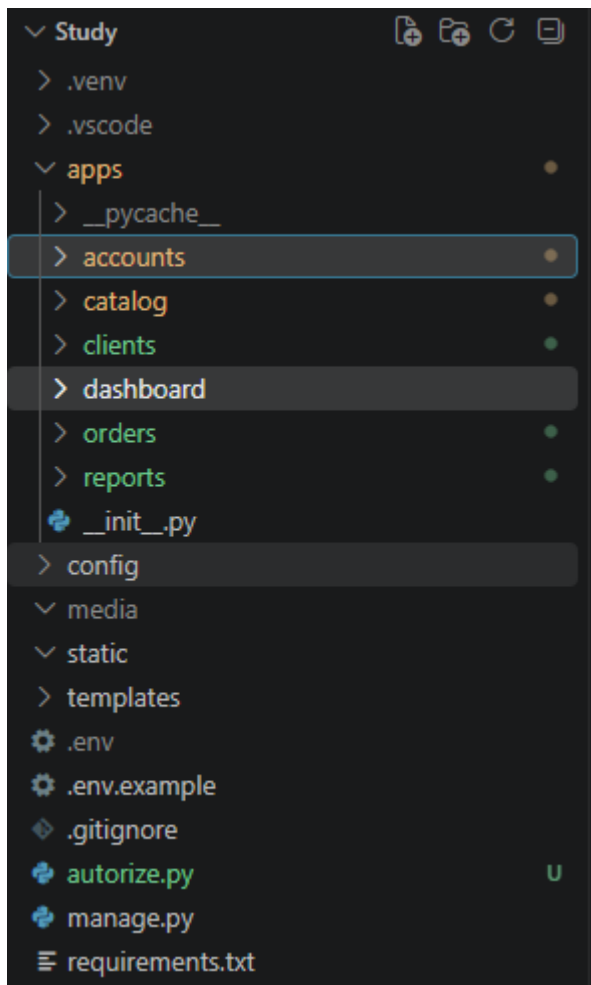
config > urls.py > ...
1  from django.contrib import admin
2  from django.urls import path, include
3  from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
4
5  urlpatterns = [
6      # Админка Django (HTML только здесь, но это готовое)
7      path("admin/", admin.site.urls),
8
9      # === МОДУЛЬ 1: АВТОРИЗАЦИЯ ===
10     path("api/auth/", include("apps.accounts.urls")),
11
12     # === МОДУЛЬ 2: КАТАЛОГ ===
13     path("api/", include("apps.catalog.urls")),
14
15     # === МОДУЛЬ 3: КЛИЕНТЫ ===
16     path("api/clients/", include("apps.clients.urls")),
17
18     # === МОДУЛЬ 4: ЗАКАЗЫ ===
19     path("api/orders/", include("apps.orders.urls")),
20
21     # === МОДУЛЬ 5: ОТЧЁТЫ ===
22     path("api/reports/", include("apps.reports.urls")),
23
24     # DRF-вход через браузер (для тестирования)
25     path("api/drf-login/", include("rest_framework.urls")),
26 ]

```

Создать serializers.py в apps/accounts/



dashboard/ в корневой папке



Обновляем config/settings.py

```

config > settings.py > ...
1  from pathlib import Path
2  from datetime import timedelta
3  import os
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  BASE_DIR = Path(__file__).resolve().parent.parent
9
10 SECRET_KEY = os.getenv("SECRET_KEY", "django-insecure-change-me")
11 DEBUG = True
12 ALLOWED_HOSTS = ["*"]
13
14 # ПРИЛОЖЕНИЯ
15
16 INSTALLED_APPS = [
17     "django.contrib.admin",
18     "django.contrib.auth",
19     "django.contrib.contenttypes",
20     "django.contrib.sessions",
21     "django.contrib.messages",
22     "django.contrib.staticfiles",
23
24     # DRF и вспомогательные
25     "rest_framework",
26     "rest_framework_simplejwt",
27     "corsheaders",
28     "django_filters",
29
30     # НАШИ 5 МОДУЛЕЙ
31     "apps.accounts",
32     "apps.catalog",
33     "apps.clients",
34     "apps.orders",
35     "apps.reports",
36 ]
37
38 MIDDLEWARE = [
39     "corsheaders.middleware.CorsMiddleware", # CORS первым
40     "django.middleware.security.SecurityMiddleware",
41     "django.contrib.sessions.middleware.SessionMiddleware",
42     "django.middleware.common.CommonMiddleware",
43     "django.middleware.csrf.CsrfViewMiddleware",
44     "django.contrib.auth.middleware.AuthenticationMiddleware",
45     "django.contrib.messages.middleware.MessageMiddleware",
46     "django.middleware.clickjacking.XFrameOptionsMiddleware",
47 ]
48
49 ROOT_URLCONF = "config.urls"
50
51 # ШАБЛОНЫ
52
53 TEMPLATES = [
54     {
55         "BACKEND": "django.template.backends.django.DjangoTemplates",
56         "DIRS": [], # пусто, своих шаблонов нет
57         "APP_DIRS": True, # только встроенные (admin)
58         "OPTIONS": {
59             "context_processors": [
60                 "django.template.context_processors.debug",
61                 "django.template.context_processors.request",
62                 "django.contrib.auth.context_processors.auth",
63                 "django.contrib.messages.context_processors.messages",
64             ],
65         },
66     },
67 ]

```

```

67 ]
68
69 WSGI_APPLICATION = "config.wsgi.application"
70
71 # БАЗА ДАННЫХ
72
73 DATABASES = {
74     "default": {
75         "ENGINE": "django.db.backends.postgresql",
76         "NAME": os.getenv("DB_NAME", "autosalon_db"),
77         "USER": os.getenv("DB_USER", "autosalon"),
78         "PASSWORD": os.getenv("DB_PASSWORD", "autosalon_pass"),
79         "HOST": os.getenv("DB_HOST", "localhost"),
80         "PORT": os.getenv("DB_PORT", "5432"),
81     }
82 }
83
84
85 # АУТЕНТИФИКАЦИЯ
86
87 AUTH_USER_MODEL = "accounts.User"
88
89 # DRF — НАСТРОЙКИ API
90
91 REST_FRAMEWORK = {
92     # JWT по умолчанию
93     "DEFAULT_AUTHENTICATION_CLASSES": (
94         "rest_framework_simplejwt.authentication.JWTAuthentication",
95     ),
96     # Только авторизованные (кроме явно открытых)
97     "DEFAULT_PERMISSION_CLASSES": (
98         "rest_framework.permissions.IsAuthenticated",
99     ),
100     # Фильтрация и поиск
101     "DEFAULT_FILTER_BACKENDS": (
102         "django_filters.rest_framework.DjangoFilterBackend",
103         "rest_framework.filters.SearchFilter",
104         "rest_framework.filters.OrderingFilter",
105     ),
106     # Пагинация
107     "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
108     "PAGE_SIZE": 20,
109 }
110
111 # JWT — НАСТРОЙКИ ТОКЕНОВ
112
113 SIMPLE_JWT = {
114     "ACCESS_TOKEN_LIFETIME": timedelta(hours=8),
115     "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
116     "AUTH_HEADER_TYPES": ("Bearer",),
117 }
118
119
120 # CORS — РАЗРЕШАЕМ STREAMLIT И DESKTOP
121
122 CORS_ALLOW_ALL_ORIGINS = True # для разработки
123 # В production: CORS_ALLOWED_ORIGINS = ["http://localhost:8501"]
124
125
126 # ПРОЧЕЕ

```

```
126 # ПРОЧЕЕ
127
128 LANGUAGE_CODE = "ru-ru"
129 TIME_ZONE = "Europe/Moscow"
130 USE_I18N = True
131 USE_TZ = True
132
133 STATIC_URL = "static/"
134 STATIC_ROOT = BASE_DIR / "staticfiles"
135
136 MEDIA_URL = "media/"
137 MEDIA_ROOT = BASE_DIR / "media"
138
139 DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
```

apps/accounts/serializers.py

apps > accounts > serializers.py > LoginSerializer > validate

```
1  from rest_framework import serializers
2  from django.contrib.auth import authenticate
3  from .models import User
4
5
6  class UserSerializer(serializers.ModelSerializer):
7      """Сериализатор для вывода данных пользователя."""
8
9      class Meta:
10         model = User
11         fields = ["id", "username", "email", "first_name", "last_name",
12                 "phone", "position", "is_active"]
13         read_only_fields = ["id", "is_active"]
14
15
16  class RegisterSerializer(serializers.ModelSerializer):
17      """Сериализатор регистрации."""
18      password = serializers.CharField(write_only=True, min_length=6)
19      password2 = serializers.CharField(write_only=True, min_length=6)
20
21      class Meta:
22         model = User
23         fields = ["username", "email", "first_name", "last_name",
24                 "phone", "position", "password", "password2"]
25
26      def validate(self, data):
27         if data["password"] != data["password2"]:
28             raise serializers.ValidationError("Пароли не совпадают")
29         return data
30
31      def create(self, validated_data):
32         validated_data.pop("password2")
33         password = validated_data.pop("password")
34         user = User(**validated_data)
35         user.set_password(password)          # Хеширование
36         user.save()
37         return user
38
39
40  class LoginSerializer(serializers.Serializer):
41      """Сериализатор входа."""
42      username = serializers.CharField()
43      password = serializers.CharField(write_only=True)
44
45      def validate(self, data):
46         user = authenticate(
47             username=data["username"],
48             password=data["password"],
49         )
50         if not user:
51             raise serializers.ValidationError("Неверный логин или пароль")
52         if not user.is_active:
53             raise serializers.ValidationError("Аккаунт отключён")
54         data["user"] = user
55         return data
```

apps/catalog/serializers.py

```
apps > catalog > serializers.py > CarSerializer > Meta
1  from rest_framework import serializers
2  from .models import Brand, Car
3
4
5  class BrandSerializer(serializers.ModelSerializer):
6      class Meta:
7          model = Brand
8          fields = ["id", "name", "country"]
9
10
11  class CarSerializer(serializers.ModelSerializer):
12      brand_name = serializers.CharField(source="brand.name", read_only=True)
13      status_display = serializers.CharField(source="get_status_display", read_only=True)
14      fuel_type_display = serializers.CharField(source="get_fuel_type_display", read_only=True)
15
16      class Meta:
17          model = Car
18          fields = [
19              "id", "brand", "brand_name", "model", "year", "price",
20              "color", "engine_volume", "fuel_type", "fuel_type_display",
21              "transmission", "mileage", "status", "status_display",
22              "vin", "description", "created_at",
23          ]
24      read_only_fields = ["created_at"]
```

apps/clients/serializers.py

```
apps > clients > serializers.py > ClientSerializer > validate_phone
1  from rest_framework import serializers
2  from .models import Client
3
4
5  class ClientSerializer(serializers.ModelSerializer):
6      manager_name = serializers.CharField(
7          source="manager.get_full_name", read_only=True,
8      )
9
10     class Meta:
11         model = Client
12         fields = [
13             "id", "full_name", "phone", "email",
14             "passport", "birth_date", "manager", "manager_name", "created_at",
15         ]
16         read_only_fields = ["created_at"]
17
18     def validate_phone(self, value):
19         """Проверка уникальности телефона."""
20         qs = Client.objects.filter(phone=value)
21         if self.instance:
22             qs = qs.exclude(pk=self.instance.pk)
23         if qs.exists():
24             raise serializers.ValidationError("Клиент с таким телефоном уже есть")
25         return value
```

apps/orders/serializers.py

```
apps > orders > serializers.py > OrderSerializer > Meta
1  from rest_framework import serializers
2  from .models import Order
3
4
5  class OrderSerializer(serializers.ModelSerializer):
6      client_name = serializers.CharField(source="client.full_name", read_only=True)
7      car_name = serializers.CharField(source="car.__str__", read_only=True)
8      manager_name = serializers.CharField(source="manager.get_full_name", read_only=True)
9      status_display = serializers.CharField(source="get_status_display", read_only=True)
10     order_type_display = serializers.CharField(source="get_order_type_display", read_only=True)
11
12     class Meta:
13         model = Order
14         fields = [
15             "id", "client", "client_name", "car", "car_name",
16             "manager", "manager_name", "order_type", "order_type_display",
17             "total_amount", "status", "status_display",
18             "comment", "created_at", "completed_at",
19         ]
20         read_only_fields = ["manager", "created_at", "completed_at"]
```

STREAMLIT-ДАШБОРД (dashboard/app.py)

```

dashboard > app.py > ...
1  import os
2  import streamlit as st
3  import pandas as pd
4  import plotly.express as px
5  import requests
6
7  API_URL = os.getenv("API_URL", "http://localhost:8000")
8
9  st.set_page_config(page_title="Автосалон — Аналитика", page_icon="🚗", layout="wide")
10 st.title("🚗 Автосалон — Панель руководителя")
11
12 # === БОКОВАЯ ПАНЕЛЬ ===
13 with st.sidebar:
14     st.header("⚙️ Фильтры")
15     days = st.selectbox("Период", [7, 30, 90, 365], index=1,
16                           format_func=lambda x: f"{x} дней")
17     if st.button("🔄 Обновить"):
18         st.cache_data.clear()
19         st.rerun()
20
21
22 # ФУНКЦИИ ЗАГРУЗКИ ДАННЫХ ИЗ API
23 @st.cache_data(ttl=60)
24 def load_kpi(days):
25     return requests.get(f"{API_URL}/api/reports/kpi/", params={"days": days}).json()
26
27
28 @st.cache_data(ttl=60)
29 def load_revenue(days):
30     data = requests.get(f"{API_URL}/api/reports/revenue/", params={"days": days}).json()
31     return pd.DataFrame(data)
32
33
34 @st.cache_data(ttl=60)
35 def load_top_models():
36     return pd.DataFrame(requests.get(f"{API_URL}/api/reports/top-models/").json())
37
38
39 @st.cache_data(ttl=60)
40 def load_manager_load(days):
41     return pd.DataFrame(requests.get(f"{API_URL}/api/reports/manager-load/", params={"days": days}).json())
42
43
44 @st.cache_data(ttl=60)
45 def load_stock():
46     return pd.DataFrame(requests.get(f"{API_URL}/api/reports/stock/").json())
47
48
49 # КРІ-ПЛИТКИ
50 kpi = load_kpi(days)
51 c1, c2, c3, c4 = st.columns(4)
52 c1.metric("💰 Выручка", f"{kpi['revenue']:.0f} ₽")
53 c2.metric("✅ Продаж", kpi["sales_count"])
54 c3.metric("🧾 Средний чек", f"{kpi['avg_check']:.0f} ₽")
55 c4.metric("🚗 📦 наличии", kpi["in_stock"])
56
57 # ГРАФИК ВЫРУЧКИ
58 st.header("📈 Динамика выручки")
59 revenue_df = load_revenue(days)
60 if not revenue_df.empty:
61     fig = px.line(revenue_df, x="date", y="total", markers=True)
62     st.plotly_chart(fig, use_container_width=True)
63
64 # 2 КОЛОНКИ
65 col1, col2 = st.columns(2)
66

```

```

64 # 2 КОЛОНКИ
65 col1, col2 = st.columns(2)
66
67 with col1:
68     st.header("🏆 Топ-5 моделей")
69     top = load_top_models()
70     if not top.empty:
71         fig = px.bar(top, x="count", y="model", color="brand", orientation="h")
72         st.plotly_chart(fig, use_container_width=True)
73
74 with col2:
75     st.header("👤 Загрузка менеджеров")
76     managers = load_manager_load(days)
77     if not managers.empty:
78         fig = px.pie(managers, values="orders", names="manager")
79         st.plotly_chart(fig, use_container_width=True)
80
81 # СКЛАД
82 st.header("📦 Состояние склада")
83 stock = load_stock()
84 if not stock.empty:
85     fig = px.pie(stock, values="count", names="status",
86                 color_discrete_map={"available": "green", "reserved": "orange", "sold": "red"})
87     st.plotly_chart(fig, use_container_width=True)

```

Устанавливаем библиотеки

```

(.venv) PS C:\Study> pip install djangorestframework==3.14.0
>>
Collecting djangorestframework==3.14.0
  Downloading djangorestframework-3.14.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: django>=3.0 in .\venv\Lib\site-packages (from djangorestframework==3.14.0)
Collecting pytz (from djangorestframework==3.14.0)
  Downloading pytz-2026.4-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: asgiref>=3.9.1 in .\venv\Lib\site-packages (from django>=3.0)
Requirement already satisfied: sqlparse>=0.5.0 in .\venv\Lib\site-packages (from django>=3.0)
Requirement already satisfied: tzdata in .\venv\Lib\site-packages (from django>=3.0)
Downloading djangorestframework-3.14.0-py3-none-any.whl (1.1 MB)
----- 0.0/1.1 MB ? eta -:--:--

```

```

(.venv) PS C:\Study> pip install djangorestframework-simplejwt==5.3.1
Collecting djangorestframework-simplejwt==5.3.1
  Downloading djangorestframework_simplejwt-5.3.1-py3-none-any.whl.metadata (4.3 kB)
Requirement already satisfied: django>=3.2 in .\venv\Lib\site-packages (from djangorestframework-simplejwt==5.3.1)
Requirement already satisfied: djangorestframework>=3.12 in .\venv\Lib\site-packages (from djangorestframework-simplejwt==5.3.1)
Collecting pyjwt<3,>=1.7.1 (from djangorestframework-simplejwt==5.3.1)
  Downloading pyjwt-2.15.0-py3-none-any.whl.metadata (3.5 kB)
Requirement already satisfied: asgiref>=3.9.1 in .\venv\Lib\site-packages (from django>=3.2)
Requirement already satisfied: sqlparse>=0.5.0 in .\venv\Lib\site-packages (from django>=3.2)
Requirement already satisfied: tzdata in .\venv\Lib\site-packages (from django>=3.2)
Successfully installed djangorestframework-simplejwt-5.3.1 pyjwt-2.15.0

```

```

(.venv) PS C:\Study> pip install django-cors-headers==4.3.1
Collecting django-cors-headers==4.3.1

```

```

(.venv) PS C:\Study> pip install django-filter==23.5
Collecting django-filter==23.5

```

```
(.venv) PS C:\Study> pip install streamlit==1.30.0
```

```
(.venv) PS C:\Study> pip install pandas==2.2.0
```

```
(.venv) PS C:\Study> pip install plotly==5.18.0  
Collecting plotly==5.18.0  
  Downloading plotly-5.18.0-py3-none-any.whl.metadata (7.0 kB)
```